Campus Café Ordering and Inventory System

Database Design — Tasks 1 through 5

Task 1 – Data Requirements (Entities, Attributes, Description)

Entity	Possible Attributes	Description
Customer	CustomerID/UnivID, Name, CustomerType (student/faculty/staff), ContactNumber	Represents the person placing an order. May be identified by name or university ID.
Employee (Cashier)	EmployeeID, Name, ContactNumber, WorkSchedule	Café staff who process orders and payments.
Product	ProductID, ProductName, Description, Price, Category, AvailabilityStatus, AvailabilityPeriod	Food/beverage items sold at the café (sandwiches, rice meals, pastries, coffee, drinks).
Order	OrderID, OrderDate, OrderTime, OrderStatus (Paid/Pending/Cancelled), TotalAmount	A transaction made by a customer, processed by one cashier.
OrderDetail	OrderDetailID, OrderID, ProductID, Quantity, SellingPriceAtOrder	Line items of an order — what was bought, how much, and at what price at the time of sale.
Payment	PaymentID, OrderID, PaymentDate, AmountPaid, PaymentMethod	Records how and when a completed order was paid for.
Supplier	SupplierID, CompanyName, ContactPerson, TelephoneNumber, Email, Address	External companies that supply ingredients to the café.
Ingredient	IngredientID, IngredientName, UnitOfMeasurement, CurrentQuantity, MinimumStockLevel	Raw materials used to prepare products (rice, coffee beans, milk, bread, meat, vegetables, sugar, etc.).
SupplierIngredient	SupplierID, IngredientID, UnitPrice	Associative entity resolving the many-to-many relationship between suppliers and ingredients.
Delivery	DeliveryID, SupplierID, DeliveryDate	Header record of a delivery event from a supplier.
DeliveryDetail	DeliveryID, IngredientID, QuantityReceived	Line items of a delivery — which ingredients and how much were received.
ProductIngredient (Recipe)	ProductID, IngredientID, QuantityRequired	Links products to the ingredients/amounts needed — explains why a product goes unavailable when an ingredient runs out.

Task 2 – Business Rules

1. A customer may place one or more orders, but each order belongs to exactly one customer (or is recorded with a name/ID at minimum).

2. An order must contain at least one product, and an order may contain many different products.
3. A product may appear in many different orders over time.
4. Each cashier can process many orders, but each order is processed by exactly one cashier.
5. Each order must have exactly one status at any given time: Paid, Pending, or Cancelled.
6. A payment is recorded only for an order that is marked as paid/completed; an order need not have a payment yet if it is still pending.
7. Each order can be associated with one payment record (assuming full, single-transaction payment); the payment records the amount, date, and method (cash, debit, credit, or approved digital payment).
8. The price of a product recorded in an order (selling price at time of purchase) must be stored independently of the product's current listed price, since prices may change over time.
9. A supplier may supply many different ingredients, and an ingredient may be supplied by more than one supplier (many-to-many relationship).
10. Each delivery is associated with exactly one supplier, but a delivery may include multiple ingredients, each with its own received quantity.
11. Each ingredient must have a minimum stock level defined so the system can flag when the ingredient is low or out of stock.
12. A product may require one or more ingredients to be prepared, and an ingredient may be used in the preparation of many different products.
13. A product must be marked unavailable if any required ingredient's current stock reaches zero, and may be time-restricted to specific availability periods.
14. Only authorized users (cashiers, inventory staff, managers) may create, update, or view records in the system, according to their role.
15. The total amount of an order must be calculated as the sum of (quantity × selling price) of all order line items.

Task 3 – Conceptual Data Model

The conceptual model focuses only on entities, relationships, and cardinalities — no keys or data types.

Entities

Customer, Employee (Cashier), Order, Product, Payment, Supplier, Ingredient, Delivery

Relationships and Cardinalities

```
Customer        (1) -----< (M)  Order
Employee        (1) -----< (M)  Order
Order           (1) -----< (M)  Payment           [an order can have payment records once completed]
Order           (M) >-----< (M) Product            [resolved by OrderDetail - "contains"]
Supplier        (M) >-----< (M) Ingredient         [resolved by SupplierIngredient - "supplies"]
Supplier        (1) -----< (M)  Delivery
Delivery        (M) >-----< (M) Ingredient         [resolved by DeliveryDetail - "receives"]
Product         (M) >-----< (M) Ingredient         [resolved by ProductIngredient - "requires"]
```

Narrative Summary of Relationships

- A Customer places many Orders; each Order belongs to one Customer.
- An Employee (Cashier) processes many Orders; each Order is processed by one Employee.
- An Order contains many Products, and a Product can appear in many Orders (many-to-many, via order line items).

- An Order is settled by a Payment once completed.
- A Supplier supplies many Ingredients, and an Ingredient can be supplied by many Suppliers (many-to-many).
- A Supplier makes many Deliveries; each Delivery comes from one Supplier.
- A Delivery can include many Ingredients, and an Ingredient can appear in many Deliveries (many-to-many, via delivery line items).
- A Product requires many Ingredients, and an Ingredient can be used by many Products (many-to-many recipe relationship).

Task 4 – Logical Data Model

At this stage, many-to-many relationships are resolved into associative (junction) entities, and primary/foreign keys are identified. Still DBMS-independent (no data types).

Entities and Keys

Customer

- PK: CustomerID
- Name, ContactNumber, CustomerType

Employee

- PK: EmployeeID
- Name, ContactNumber, WorkSchedule

Product

- PK: ProductID
- ProductName, Description, Price, Category, AvailabilityStatus, AvailabilityPeriod

Order

- PK: OrderID
- FK: CustomerID -> Customer
- FK: EmployeeID -> Employee
- OrderDate, OrderTime, OrderStatus, TotalAmount

OrderDetail (associative: Order <-> Product, M:N)

- PK: OrderDetailID (or composite PK: OrderID + ProductID)
- FK: OrderID -> Order
- FK: ProductID -> Product
- Quantity, SellingPriceAtOrder

Payment

- PK: PaymentID
- FK: OrderID -> Order
- PaymentDate, AmountPaid, PaymentMethod

Supplier

- PK: SupplierID
- CompanyName, ContactPerson, TelephoneNumber, Email, Address

Ingredient

- PK: IngredientID
- IngredientName, UnitOfMeasurement, CurrentQuantity, MinimumStockLevel

SupplierIngredient (associative: Supplier <-> Ingredient, M:N)

- PK: composite (SupplierID + IngredientID)
- FK: SupplierID -> Supplier

- FK: IngredientID -> Ingredient
- UnitPrice

Delivery

- PK: DeliveryID
- FK: SupplierID -> Supplier
- DeliveryDate

DeliveryDetail (associative: Delivery <-> Ingredient, M:N)

- PK: composite (DeliveryID + IngredientID)
- FK: DeliveryID -> Delivery
- FK: IngredientID -> Ingredient
- QuantityReceived

ProductIngredient (associative: Product <-> Ingredient, Recipe)

- PK: composite (ProductID + IngredientID)
- FK: ProductID -> Product
- FK: IngredientID -> Ingredient
- QuantityRequired

Relationship Summary (Logical Level)

Parent	Child	Cardinality
Customer	Order	1 : M
Employee	Order	1 : M
Order	OrderDetail	1 : M
Product	OrderDetail	1 : M
Order	Payment	1 : M (typically 1:1 in practice)
Supplier	SupplierIngredient	1 : M
Ingredient	SupplierIngredient	1 : M
Supplier	Delivery	1 : M
Delivery	DeliveryDetail	1 : M
Ingredient	DeliveryDetail	1 : M
Product	ProductIngredient	1 : M
Ingredient	ProductIngredient	1 : M

Task 5 – Physical Data Model (MySQL DDL)

```
CREATE DATABASE campus_cafe_db;
USE campus_cafe_db;

-- ============================
-- CUSTOMER
-- ============================
CREATE TABLE Customer (
    CustomerID      INT AUTO_INCREMENT PRIMARY KEY,
    UnivID          VARCHAR(20)      NULL,
    Name            VARCHAR(100)     NOT NULL,
```

```
    ContactNumber   VARCHAR(20)      NULL,
    CustomerType    ENUM('Student','Faculty','Employee','Other') NOT NULL
);

-- ============================
-- EMPLOYEE (CASHIER)
-- ============================
CREATE TABLE Employee (
    EmployeeID      INT AUTO_INCREMENT PRIMARY KEY,
    Name            VARCHAR(100)     NOT NULL,
    ContactNumber   VARCHAR(20)      NULL,
    WorkSchedule    VARCHAR(100)     NULL
);

-- ============================
-- PRODUCT
-- ============================
CREATE TABLE Product (
    ProductID           INT AUTO_INCREMENT PRIMARY KEY,
    ProductName         VARCHAR(100)     NOT NULL,
    Description          VARCHAR(255)     NULL,
    Price                DECIMAL(10,2)    NOT NULL,
    Category             VARCHAR(50)      NOT NULL,
    AvailabilityStatus   ENUM('Available','Unavailable') NOT NULL DEFAULT 'Available',
    AvailabilityPeriod   VARCHAR(50)      NULL
);

-- ============================
-- SUPPLIER
-- ============================
CREATE TABLE Supplier (
    SupplierID       INT AUTO_INCREMENT PRIMARY KEY,
    CompanyName      VARCHAR(100)     NOT NULL,
    ContactPerson    VARCHAR(100)     NULL,
    TelephoneNumber  VARCHAR(20)      NULL,
    Email            VARCHAR(100)     NULL,
    Address          VARCHAR(255)     NULL
);

-- ============================
-- INGREDIENT
-- ============================
CREATE TABLE Ingredient (
    IngredientID        INT AUTO_INCREMENT PRIMARY KEY,
    IngredientName       VARCHAR(100)     NOT NULL,
    UnitOfMeasurement    VARCHAR(20)      NOT NULL,
    CurrentQuantity       DECIMAL(10,2)    NOT NULL DEFAULT 0,
    MinimumStockLevel     DECIMAL(10,2)    NOT NULL DEFAULT 0
);

-- ============================
-- ORDER
-- ============================
CREATE TABLE `Order` (
    OrderID        INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID     INT              NOT NULL,
    EmployeeID     INT              NOT NULL,
    OrderDate      DATE             NOT NULL,
    OrderTime      TIME             NOT NULL,
    OrderStatus    ENUM('Pending','Paid','Cancelled') NOT NULL DEFAULT 'Pending',
    TotalAmount    DECIMAL(10,2)    NOT NULL DEFAULT 0,
    CONSTRAINT fk_order_customer FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    CONSTRAINT fk_order_employee FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)
);
```

```
-- ============================
-- ORDER DETAIL (Order <-> Product)
-- ============================
CREATE TABLE OrderDetail (
    OrderDetailID       INT AUTO_INCREMENT PRIMARY KEY,
    OrderID              INT              NOT NULL,
    ProductID            INT              NOT NULL,
    Quantity              INT              NOT NULL,
    SellingPriceAtOrder   DECIMAL(10,2)    NOT NULL,
    CONSTRAINT fk_od_order   FOREIGN KEY (OrderID)   REFERENCES `Order`(OrderID),
    CONSTRAINT fk_od_product FOREIGN KEY (ProductID) REFERENCES Product(ProductID),
    UNIQUE KEY uq_order_product (OrderID, ProductID)
);

-- ============================
-- PAYMENT
-- ============================
CREATE TABLE Payment (
    PaymentID       INT AUTO_INCREMENT PRIMARY KEY,
    OrderID          INT              NOT NULL,
    PaymentDate      DATETIME         NOT NULL,
    AmountPaid       DECIMAL(10,2)    NOT NULL,
    PaymentMethod    ENUM('Cash','Debit Card','Credit Card','Digital Payment') NOT NULL,
    CONSTRAINT fk_payment_order FOREIGN KEY (OrderID) REFERENCES `Order`(OrderID)
);

-- ============================
-- SUPPLIER INGREDIENT (Supplier <-> Ingredient)
-- ============================
CREATE TABLE SupplierIngredient (
    SupplierID     INT              NOT NULL,
    IngredientID    INT              NOT NULL,
    UnitPrice        DECIMAL(10,2)    NULL,
    PRIMARY KEY (SupplierID, IngredientID),
    CONSTRAINT fk_si_supplier   FOREIGN KEY (SupplierID)   REFERENCES Supplier(SupplierID),
    CONSTRAINT fk_si_ingredient FOREIGN KEY (IngredientID) REFERENCES Ingredient(IngredientID)
);

-- ============================
-- DELIVERY
-- ============================
CREATE TABLE Delivery (
    DeliveryID     INT AUTO_INCREMENT PRIMARY KEY,
    SupplierID      INT              NOT NULL,
    DeliveryDate     DATE             NOT NULL,
    CONSTRAINT fk_delivery_supplier FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);

-- ============================
-- DELIVERY DETAIL (Delivery <-> Ingredient)
-- ============================
CREATE TABLE DeliveryDetail (
    DeliveryID       INT              NOT NULL,
    IngredientID      INT              NOT NULL,
    QuantityReceived   DECIMAL(10,2)    NOT NULL,
    PRIMARY KEY (DeliveryID, IngredientID),
    CONSTRAINT fk_dd_delivery   FOREIGN KEY (DeliveryID)   REFERENCES Delivery(DeliveryID),
    CONSTRAINT fk_dd_ingredient FOREIGN KEY (IngredientID) REFERENCES Ingredient(IngredientID)
);

-- ============================
-- PRODUCT INGREDIENT (Product <-> Ingredient, Recipe)
-- ============================
```

```
CREATE TABLE ProductIngredient (
    ProductID         INT              NOT NULL,
    IngredientID        INT              NOT NULL,
    QuantityRequired      DECIMAL(10,2)    NOT NULL,
    PRIMARY KEY (ProductID, IngredientID),
    CONSTRAINT fk_pi_product    FOREIGN KEY (ProductID)    REFERENCES Product(ProductID),
    CONSTRAINT fk_pi_ingredient FOREIGN KEY (IngredientID) REFERENCES Ingredient(IngredientID)
);
```

Notes on the Physical Design

- OrderDetail, SupplierIngredient, DeliveryDetail, and ProductIngredient are all junction (associative) tables resolving many-to-many relationships identified in the conceptual model.
- SellingPriceAtOrder in OrderDetail is intentionally separate from Product.Price to preserve historical accuracy of what was actually charged, since prices can change over time.
- Order.TotalAmount can be maintained via application logic or a trigger that sums Quantity x SellingPriceAtOrder from OrderDetail.
- Low-stock alerts can be generated with a query comparing Ingredient.CurrentQuantity to Ingredient.MinimumStockLevel - no extra table is required for this business rule.
- All foreign key columns are implicitly NOT NULL unless otherwise noted, enforcing referential integrity throughout.